

JARVIS v1.0 — Master Plan (Source of Truth)

Author: Kidus Abdula | **Version:** 1.1 | **Last Updated:** 2026-04-01

🌀 Executive Summary

JARVIS is an autonomous SWE (Software Engineering) workflow system that replaces every hosted tool, runs your entire day, ships code, and scales later into sellable client instances. Built on 100% free models via OpenRouter with manual control and individual MCP integrations.

Core Philosophy: One entity to rule them all — no rate limits, no walled gardens, pure flexibility.

1. Infrastructure

VPS Specifications

Spec	Value
Provider	Hostinger
Plan	KVM-4
vCPU	4 cores
RAM	16 GB
Storage	200 GB NVMe
Bandwidth	16 TB
OS	Ubuntu 22.04 LTS
IP	187.124.45.161

Software Stack

- **Runtime:** Docker + Docker Compose (manual, full control)
- **Core:** OpenClaw (self-hosted gateway) + individual MCP servers
- **Version Control:** GitHub (jarvis-core monorepo)
- **Container OS:** Ubuntu 22.04

2. Model Routing Strategy

Primary Model Stack (via OpenRouter)

Role	Model	Free?	Use Case
------	-------	-------	----------

Role	Model	Free?	Use Case
Orchestrator	NVIDIA Nemotron-3-Super:free	☒	Planning, audit, master docs, scope adherence, multi-agent coherence
Implementation	Nemotron-3-Super (or MiMo-V2-Pro while free)	☒	Architecture, production engineering, agentic tasks
Heavy SWE	MiniMax M2.5:free	☒	SWE-Bench tasks, office automation (Word/Excel/PPT), tool-heavy steps
Fast Execution	GLM-5-Turbo	Paid	Long-chain tool use, persistent tasks, speed-critical operations

Fallback Chain

Nemotron-3-Super:free → GLM-5-Turbo → MiniMax-M2.5:free → Step-3.5-free → Grok

OpenRouter Credit Strategy (\$20 initial)

- Default to free models (MiniMax + Nemotron have massive free quotas)
- Only burn credit on premium bursts (e.g., one-off Claude Opus audit)
- In Cline/Kilo/Roo: Set model router fallback chain
- Enable "allow prompt training" for free tier

3. Interfaces & Channels

Channel	Status	Use Case
Telegram	☒ Active	Primary chat interface (@Jarvis1015Bot)
WhatsApp	☐ Pending	Client communication
Web Dashboard	☒ Active	Control panel (port 18789)
Gmail	☐ Pending	Email automation (OAuth setup required)
Notion	☒ Configured	Documentation, knowledge base
GitHub	☒ Configured	Code management, CI/CD
Vercel	☒ Configured	Deployment
VS Code	☐ Pending	Workspace integration

4. MCP Integrations (Individual — Full Control)

MCP Server Status

Server	Status	Config	Purpose
--------	--------	--------	---------

Server	Status	Config	Purpose
GitHub MCP	☒ Configured	config/mcp-servers/github.yaml	Repository management, PRs, issues, commits
Vercel MCP	☒ Configured	config/mcp-servers/vercel.yaml	Deployment, project management
Notion MCP	☒ Configured	config/mcp-servers/notion.yaml	Documentation, master docs, knowledge base
Browser MCP	☒ Configured	config/mcp-servers/browser.yaml	Web automation, research
Gmail MCP	☐ Pending OAuth	config/mcp-servers/gmail.yaml	Email, calendar, drive
VS Code MCP	☐ Pending	TBD	Workspace integration, linter, terminal
Shell/File MCP	☐ Pending	TBD	System operations

MCP Philosophy

- Individual MCPs over Composio (pure flexibility, zero vendor lock)
- Each MCP is a separate Docker container or npx process
- Full control over each integration
- Easy to add/remove without affecting others

5. Phase Breakdown (7-Day Sprint)

Phase 1: VPS + OpenClaw + Telegram ☒ COMPLETE

- ☒ VPS base setup
- ☒ Docker installation
- ☒ OpenClaw deployment
- ☒ OpenRouter connection
- ☒ Telegram bot integration
- ☒ Nemotron-3-Super active
- ☒ Identity setup (USER.md + IDENTITY.md)

Phase 2: Monorepo + MCPs ☒ CONFIGURATIONS COMPLETE

- ☒ jarvis-core monorepo setup (local + VPS)
- ☒ OpenClaw gateway configuration (config/openclaw.json)
- ☒ GitHub MCP configuration
- ☒ Vercel MCP configuration
- ☒ Notion MCP configuration
- ☒ Browser MCP configuration

- ☒ Gmail MCP configuration (pending OAuth)
- ☒ Docker Compose updated with MCP services
- ☐ Deploy to VPS and test MCP connections
- ☐ Test unified chat interface

Phase 3: Daily Workflow Automation

- ☐ Morning audit workflow
- ☐ Plan → Code → Git → Vercel → Notion → Telegram pipeline
- ☐ Automated daily standup
- ☐ Cron-based task scheduling

Phase 4: Vision & Media

- ☐ Image generation MCPs
- ☐ Video generation capabilities
- ☐ Image recognition/vision
- ☐ Google One deep sync

Phase 5: Persistent Agents

- ☐ Long-running agent tasks
- ☐ Cron integration
- ☐ Master doc automation in Notion

Phase 6: Security & Hardening

- ☐ Security audit
- ☐ Backup automation
- ☐ Full SWE loop testing
- ☐ Performance optimization

Phase 7: Export & Monetization

- ☐ Export as template
- ☐ First client mock
- ☐ SaaS preparation
- ☐ Celebrate JARVIS live 🐛

6. Daily Driver Commands

These will live in OpenClaw chat:

Command	Description
"Jarvis, daily audit"	Run morning standup: check GitHub, Notion, email, VPS health

Command	Description
"Jarvis, ship feature X"	Full pipeline: plan → code → commit → deploy → update docs
"Jarvis, generate proposal PPT"	Create presentation from Notion data
"Jarvis, send to client on WhatsApp"	Deliver artifacts via WhatsApp
"Jarvis, run morning protocol"	Complete daily automation sequence
"Jarvis, backup config"	Backup and commit config to GitHub

7. Monetization Strategy (Future)

Sellable Products

- 1. **SWE Agency JARVIS Template** — Pre-configured autonomous SWE system
- 2. **MCP Skill Packs** — Curated integration bundles
- 3. **White-Label Instances** — Custom JARVIS for clients
- 4. **Automation Consulting** — Setup and optimization services

Target Markets

- Software development agencies
- Freelance developers
- Small dev teams
- Non-technical business owners (via templates)

Pricing Model

- **Template:** \$299-499 one-time
- **Managed Instance:** \$99-199/month
- **Custom Integration:** \$500-2000 per setup
- **Consulting:** \$150-300/hour

8. Safety & Rollback

Backup Strategy

- All configs in Git (jarvis-core monorepo)
- Docker volumes for persistent data
- VPS snapshots before each phase
- Automated backup script (daily commit to GitHub)

Rollback Procedures

```
# Stop everything
./scripts/docker-stop.sh

# Restore config from Git
git checkout HEAD~1 -- config/

# Restart
./scripts/docker-start.sh

# Nuclear option - full restore
git reset --hard <commit-hash>
docker compose down && docker compose up -d
```

9. Technology Decisions Log

Decision	Rationale	Date
OpenClaw over Composio	Full control, self-hosted, no vendor lock	2026-03-23
Individual MCPs over bundled	Pure flexibility, modular	2026-03-23
Free models via OpenRouter	Zero recurring cost, SOTA quality	2026-03-23
Docker over native install	Portable, reproducible, easy rollback	2026-03-23
Monorepo (jarvis-core)	Single source of truth, version-controlled	2026-03-23
Telegram as primary interface	Mobile-first, bot-friendly, free	2026-03-23
YAML configs for MCPs	Human-readable, easy to modify	2026-04-01

10. Version History

Version	Date	Changes
1.0	2026-03-31	Initial master plan, Phase 1 complete
1.1	2026-04-01	Phase 2 MCP configurations complete, OpenClaw config created, Docker Compose updated
1.2	TBD	Phase 2 complete (MCPs deployed and tested)
2.0	TBD	Full autonomous SWE loop

11. Worklog Summary

2026-04-01 Session

- **18:33** - Project status assessment, identified Phase 2 gap

- **19:28** - Created ONBOARDING.md, WORKLOG.md, MCP-SETUP.md
 - **20:35** - Created `config/openclaw.json` gateway configuration
 - **20:38** - Created all MCP server configs in `config/mcp-servers/`
 - **20:39** - Created MCP server README files in `mcp/*/`
 - **20:40** - Updated `docker-compose.yml` with MCP services
-

Critical Notes

1. **Never use paid hosted providers** — OpenRouter free models only
 2. **All changes go through Git** — Nothing is modified directly on VPS without commit
 3. **Test before commit** — Always verify changes work before pushing
 4. **Security first** — Keep tokens in `.env`, never in code
 5. **Document everything** — This file is the source of truth
-

This document is the source of truth for JARVIS v1.0. All decisions, configurations, and plans are documented here.